MC970 - Introdução à programação paralela
Relatório - Exercícios com CUDA

Daniel de Sousa Cipriano RA: 233228

Maio 2025

1 Descrição de sistema

Para execução dos exercícios foram utilizados serviços do Google Colab, que oferece acesso gratuito à GPUs, permite rodar código através de um ambiente interativo Jupyter Notebook e operar um terminal Linux de servidor em nuvem da Google Cloud Platform, tudo diretamente pelo navegador web[1]. A GPU selecionada para uso no projeto foi a Nvidia Tesla T4, as especificações da placa podem ser vistas nas Figuras 1 e 2, retiradas de documentos disponíveis no site oficial da Nvidia[2][3]. Também foi possível rodar o comando `nvidia-smi` na CLI do Google Colab para obter mais informações da GPU, o resultado pode ser visto na Figura 3. O ambiente Google Colab implementa notebooks de trabalho, que rodam em máquinas virtuais. A máquina utilizada no projeto opera com um sistema Linux 64 bits, 112,6 GB de memória em disco, 12,7 GB de memória RAM e 15 GB de memória VRAM.

2 Ferramentas

As principais ferramentas utilizadas no desenvolvimento do projeto foram:

CUDA: CUDA (Compute Unified Device Architecture) é um modelo de programação desenvolvido pela Nvidia para permitir a execução de tarefas não gráficas na GPU, fornecendo ferramentas de controle para que o programador possa implementar computação heterogênea, alternando a execução entre CPU e GPU. Com CUDA, é possível utilizar o poder de paralelismo das GPUs, para otimizar tarefas que necessitam de computação pesada, como por exemplo modelagem científica, análise de grandes conjuntos de dados, reconhecimento de padrões, processamento de imagens e vídeos e etc. Nesse projeto, foi utilizada a extensão CUDA C/C++, que implementa funções nas linguagens C e C++, para manipulação de código entre o host(CPU) e device(GPU), controle de sincronização, controle de memória, balanceamento de carga e etc.
Nvidia Nsight Compute: Nsight Compute é uma ferramenta de análise de desempenho desenvolvida pela Nvidia, projetada para oferecer suporte avançado à otimização de kernels CUDA, permitindo uma visão detalhada da execução de código na GPU. Com o Nsight Compute, é possível coletar métricas específicas de hardware, como ocupação dos multiprocessadores, eficiência de

memória e uso de registradores, auxiliando o programador na identificação de gargalos e oportunidades de melhoria em aplicações paralelas. A ferramenta fornece uma interface interativa e também suporte por linha de comando, permitindo a integração com diferentes fluxos de trabalho[4]. Nesse projeto o Nsight Compute foi utilizado para analisar a ocupação de warps CUDA.

3 Resumo do funcionamento básico de códigos CUDA

O princípio básico do modelo CUDA é alternar a execução de código entre CPU e GPU. Para isso, são oferecidos qualificadores de função, são eles:

`__global__`: função de kernel chamada pela CPU e executada na GPU. Deve retornar void.
`__device__`: função auxiliar chamada na GPU e executada na GPU. Pode retornar variados tipos.
`__host__`: função chamada na CPU e executada na CPU, é o padrão utilizado e o qualificador pode ser omitido.

A função `__global__` é chamada no host seguindo o formato `__global__<<b, t>>(a, b)`, no qual b representa o número de blocos de threads, t representa a quantidade de threads em cada bloco, a e b representam os argumentos da função. Como a CPU utiliza a memória principal do sistema e a GPU utiliza sua própria memória dedicada, ponteiros para memória definidos no host precisam ser convertidos para memória do device. Essa conversão é feita utilizando as funções `cudaMalloc` que aloca memória no device, e a função `cudaMemcpy` que copia memória do host para o device ou do device para o host, de acordo com a flag passada como argumento na função.

O hardware de GPUs com suporte para CUDA, divide a arquitetura em três partes, são elas:

CUDA Core: implementa um núcleo CUDA que pode executar uma thread.
SM(Streaming Multiprocessor): bloco que contém vários CUDA Cores.
GPU: Conjunto de vários SMs, chamado de grid.

Cada bloco de threads CUDA, é subdividido em partes chamadas de warps, um warp possui 32 threads e pode executá-las de forma independente. As GPUs com CUDA seguem a arquitetura SIMT(Single Instruction Multiple Threads), na qual cada SM recebe um bloco de threads e possui escalonadores de warps, que selecionam um warp para executar uma instrução sobre todas suas threads. Como cada warp suporta 32 threads, é recomendado que a quantidade de threads por blocos seja definida como um múltiplo de 32. O escalonador de warps trabalha buscando warps disponíveis para execução, se não existirem warps suficientes, a GPU pode ficar ociosa, o que pode levar a redução de desempenho. Portanto, é preferível manter um valor alto de ocupação, mesmo que isso não garanta necessariamente uma performance alta. A proporção de ocupação da GPU é medida pela divisão do número de warps ativos pelo número total de warps.

O modelo CUDA oferece atributos para gerenciar tamanhos de grid, bloco e índices de threads, são eles:

`gridDim.x`: Tamanho do grid(total de blocos) em dimensão x.
`gridDim.y`: Tamanho do grid(total de blocos) em dimensão y.
`gridDim.z`: Tamanho do grid(total de blocos) em dimensão z.
`blockDim.x`: Tamanho do bloco(total de threads) em dimensão x.
`blockDim.y`: Tamanho do bloco(total de threads) em dimensão y.
`blockDim.z`: Tamanho do bloco(total de threads) em dimensão z.
`blockId.x`: Índice do bloco na grid em dimensão x.
`blockId.y`: Índice do bloco na grid em dimensão y.
`blockId.z`: Índice do bloco na grid em dimensão z.
`threadId.x`: Índice da thread no bloco em dimensão x.
`threadId.y`: Índice da thread no bloco em dimensão y.
`threadId.z`: Índice da thread no bloco em dimensão z.

4 Método de balanceamento de carga utilizado nos exercícios

Como citado anteriormente, é necessário definir uma configuração de grid ideal, para que haja aproveitamento máximo do poder de paralelismo da GPU. Para exemplificar o impacto gerado pelas variações de dimensão, foram adotadas quatro abordagens distintas. Considerando que o número máximo de threads por bloco que o CUDA suporta é de 1024 threads, a GPU Nvidia T4 possui 2560 CUDA Cores, divididos em 40 SMs(64 CUDA cores por SM) e o número ideal de blocos baseado no número de threads pode ser dado pela expressão:

$$B = \frac{N + T - 1}{T} \tag{1}$$

Onde B é o número de blocos, N é o tamanho do conjunto de dados e T é o número de threads. Foram analisadas as seguintes situações:

32 blocos, 512 threads por bloco: número de blocos menor que de SMs, número médio de threads por bloco, 16 warps por bloco.
Número de blocos baseado na Equação 1, 64 threads por bloco: poucas threads por bloco, 2 warps por bloco.
Número de blocos baseado na Equação 1, 512 threads por bloco: número médio de threads por bloco, 16 warps por bloco.
Número de blocos baseado na Equação 1, 1024 threads por bloco: máximo de threads por bloco, 32 warps por bloco.

Essa comparação visa encontrar a melhor relação entre paralelismo, escalabilidade e uso eficiente dos recursos da GPU.

5 CUDA: Matrix Sum

5.1 Objetivo

O objetivo desse exercício foi paralelizar com CUDA, um código em linguagem C que implementa um algoritmo de soma de matrizes. A versão serial do programa recebe dois valores de entrada, que representam o número de linhas e colunas das matrizes a serem somadas, implementa as matrizes em forma de array com dimensão linhas x colunas e calcula a matriz resultante da soma, através do método mais simples, com loops que percorrem cada dupla de elementos a serem somados. A versão serial do código retorna a soma de todos os elementos da matriz resultante e o tempo de execução da soma de matrizes.

5.2 Resultados

Para implementar a versão paralela do programa, foi utilizada a função de kernel `__global__ void matrix_sum`. Essa função recebe os ponteiros para array de matrizes, com endereços convertidos para a memória da GPU, e a dimensão linhas x colunas das matrizes. Cada thread é selecionada pelo seu índice no grid de blocos e executa a soma para as posições equivalentes das matrizes. O resultado é armazenado na posição equivalente da matriz resultante. Essa lógica foi incluída dentro de um loop que testa se o índice de thread ultrapassou o número de elementos das matrizes. Caso haja menos threads do que posições, o índice é atualizado para continuar processando a partir da posição em que a última thread parou.

Foram analisados tempos de execução da versão serial e paralela, para gerar gráficos de desempenho, utilizando a biblioteca `matplotlib` em um script Python. A medição do tempo de execução foi delimitada à porção de código com maior overhead nos programas, já que a medição do tempo de execução total não reflete o ganho real de desempenho com paralelismo de GPU, pois adiciona o overhead das funções de preparação de ambiente CUDA. Em alguns momentos, os eixos dos gráficos foram representados em escala logarítmica, para facilitar a visualização.

Analisando o gráfico na Figura 4, que mostra o speedup por tamanho do array de matriz para as quatro configurações abordadas, vemos que a configuração com 512 threads e número de blocos definido pela Equação 1 se destaca com maior speedup. Podemos ver que a utilização de poucas threads na configuração com 64 threads e a utilização de muitas threads na configuração com 1024 threads, reduz o desempenho, tanto pelo aproveitamento reduzido de paralelismo, com recursos sendo subutilizados pelas threads, quanto pelo aproveitamento exagerado de paralelismo, com recursos se tornando escassos e concorridos entre threads. Vemos que a configuração com número de blocos menor que o número de warps possui o menor desempenho, refletindo a grande perca de poder de paralelismo que essa configuração gera. Olhando o gráfico de eficiência por tamanho do array de matriz na Figura 5, vemos a configuração com 64 threads se destacando. Esse comportamento pode ocorrer devido ao número reduzido de threads, mas não reflete necessariamente melhor performance. O gráfico de tempo de execução serial e paralela para as quatro configurações pode ser visto na Figura 6. Podemos ver o reflexo dessas métricas na análise de ocupação de warps feita com Nsight Compute, nas Figuras 7, 8, 9 e 10.

6 CUDA: Histogram

6.1 Objetivo

O objetivo desse exercício foi paralelizar um código em linguagem C, que gera um histograma de proporção das cores vermelho, verde e azul, se baseando em uma imagem de entrada em formato PPM(Portable Pixmap). O formato de imagem PPM gera um arquivo simples e sem compressão, ele define uma estrutura de arquivo que contém o valor de altura e largura da matriz de pixels da imagem e os dados binários que representam valores de cor para cada pixel, seguindo o padrão RGB(red, green, blue). O arquivo também contém o valor máximo que cada canal RGB pode atingir. O programa recebe uma imagem PPM como entrada e extrai essas informações, construindo duas estruturas. A principal definida como `PPMImage`, guarda a altura e largura da matriz de pixels e um array de matriz, com dimensão altura x largura. Cada posição do array representa um elemento da estrutura `PPMPixel`, que guarda os valores RGB de cada pixel da imagem, variando até um máximo de 256(8 bits). Posteriormente, esses valores RGB são normalizados e passam a representar um valor proporcional em ponto flutuante que varia entre 0 e 3. O programa serial testa todas as possibilidades de RGB entre 0 e 3, para todos os pixels do array de matriz, de forma sequencial. No fim, é calculado o número de ocorrências para cada valor RGB, o resultado é normalizado para representar a proporção de ocorrências do valor RGB e inserido em um array de histograma. O programa retorna os valores do histograma e o tempo de execução do trecho de código que calcula o histograma.

6.2 Resultados

Para implementar a versão paralela do programa, foi utilizada a função de kernel `__global__ void histogram_kernel`. Essa função recebe os ponteiros para o array de pixels e para o array de histograma, com endereço convertido para a memória da GPU, e a dimensão altura x largura da imagem de entrada. Cada thread da GPU é designada para lidar com um pixel equivalente ao seu índice de thread e incrementar a posição do histograma que armazena o número de ocorrências do valor RGB normalizado desse pixel. Como o array de histograma é compartilhado e incrementado paralelamente entre threads, é necessário utilizar a função `atomicAdd` do CUDA, para garantir sincronia e evitar condições de corrida. Como no exercício anterior, a lógica paralela está inserida dentro de um loop que testa se o índice de thread ultrapassou o número de elementos do array de pixels e se existirem menos threads do que pixels, é definido um novo índice para continuar da posição em que a última thread parou.

Analisando o gráfico de speedup por tamanho do array de matriz de pixels na Figura 11, vemos o speedup partir de um valor alto para entradas menores, decair para entradas médias e voltar a subir para entradas maiores. Esse comportamento pode ser explicado a partir da relação entre o tamanho da matriz de pixels e o aproveitamento efetivo da GPU durante a execução dos kernels CUDA. Para matrizes pequenas, a GPU apresenta alto desempenho devido ao baixo custo de agendamento, à rápida execução dos blocos e à baixa latência de acesso à memória. No entanto, à medida que o tamanho da matriz aumenta para uma faixa média, observa-se uma queda no speedup, que pode estar associada à subutilização dos multiprocessadores (SMs), ao aumento da competição por recursos como registradores e memória compartilhada, e ao impacto relativo dos overheads fixos de

inicialização. Para matrizes muito grandes, o tempo de execução se torna suficientemente longo para reduzir os custos fixos, permitindo que a GPU opere com alta ocupação e aproveitamento do paralelismo massivo. Nesse caso, vemos que a configuração com 512 threads e número de blocos definidos pela Equação 1, é ligeiramente superior às outras, porém não se destaca tanto quanto no exercício anterior. Olhando para o gráfico de eficiência por tamanho do array de matriz de pixels na Figura 12, novamente temos a configuração com 64 threads se destacando, devido ao reduzido número de threads, o que não indica maior aproveitamento do paralelismo de GPU. O gráfico de tempo de execução serial e paralela pode ser visto na Figura 13. Nesse caso, os tempos de execução das diferentes versões paralelizadas não foram tão divergentes entre eles. As métricas de ocupação geradas com Nsight Compute para as diferentes configurações nesse exercício, podem ser vistas nas Figuras 14, 15, 16 e 17.

Apêndice



SPECIFICATIONS

GPU Architecture	**NVIDIA Turing**
NVIDIA Turing Tensor Cores	**320**
NVIDIA CUDA® Cores	**2,560**
Single-Precision	**8.1 TFLOPS**
Mixed-Precision (FP16/FP32)	**65 TFLOPS**
INT8	**130 TOPS**
INT4	**260 TOPS**
GPU Memory	**16 GB GDDR6 300 GB/s**
ECC	**Yes**
Interconnect Bandwidth	**32 GB/sec**
System Interface	**x16 PCIe Gen3**
Form Factor	**Low-Profile PCIe**
Thermal Solution	**Passive**
Compute APIs	**CUDA, NVIDIA TensorRT™, ONNX**

Figura 1: Especificações da GPU Nvidia Tesla T4

Table 5. Comparison of the Pascal Tesla P4 and the Turing Tesla T4

GPU	Tesla P4 (Pascal)	Tesla T4 (Turing)
GPCs	4	5
TPCs	20	20
SMs	20	40
CUDA Cores/SM	128	64
CUDA Cores/GPU	2,560	2,560
Tensor Cores/SM	NA	8
Tensor Cores/GPU	NA	320
RT Cores	NA	40
GPU Base Clock MHz	810	585*
GPU Boost Clock MHz	1,063	1,590
Peak FP32 TFLOPS	5.5	8.1
Peak INT32 TIPS	NA	8.1
Peak FP16 TFLOPS	NA	16.2
Peak FP16 Tensor TFLOPS with FP16 Accumulate*	NA	65
Peak FP16 Tensor TFLOPS with FP32 Accumulate*	NA	65
Peak INT8 Tensor TOPS*	22	130
Peak INT4 Tensor TOPS*	NA	260
Frame Buffer Memory Size and Type	8192 MB GDDR5X	16384 MB GDDR6
Memory Interface	256-bit	256-bit
Memory Clock (Data Rate)	6 Gbps	10 Gbps
Memory Bandwidth (GB/sec)	192	320
ROPs	64	64
TDP	75 Watts	70 Watts

Figura 2: Especificações da GPU Nvidia Tesla T4, comparadas às especificações da GPU Nvidia Tesla P4

```
+-----------------------------------------------------------------------------------------+
| NVIDIA-SMI 550.54.15              Driver Version: 550.54.15      CUDA Version: 12.4     |
|-----------------------------------------+------------------------+----------------------+
| GPU  Name                 Persistence-M | Bus-Id          Disp.A | Volatile Uncorr. ECC |
| Fan  Temp   Perf          Pwr:Usage/Cap |           Memory-Usage | GPU-Util  Compute M. |
|                                         |                        |               MIG M. |
|=========================================+========================+======================|
|   0  Tesla T4                       Off |   00000000:00:04.0 Off |                    0 |
| N/A   43C    P8             9W /   70W |       0MiB /  15360MiB |      0%      Default |
|                                         |                        |                  N/A |
+-----------------------------------------+------------------------+----------------------+

+-----------------------------------------------------------------------------------------+
| Processes:                                                                              |
|  GPU   GI   CI        PID   Type   Process name                              GPU Memory |
|        ID   ID                                                               Usage      |
|=========================================================================================|
|  No running processes found                                                             |
+-----------------------------------------------------------------------------------------+
```

Figura 3: Resultado do comando nvidia-smi rodado na CLI do notebook Google Colab

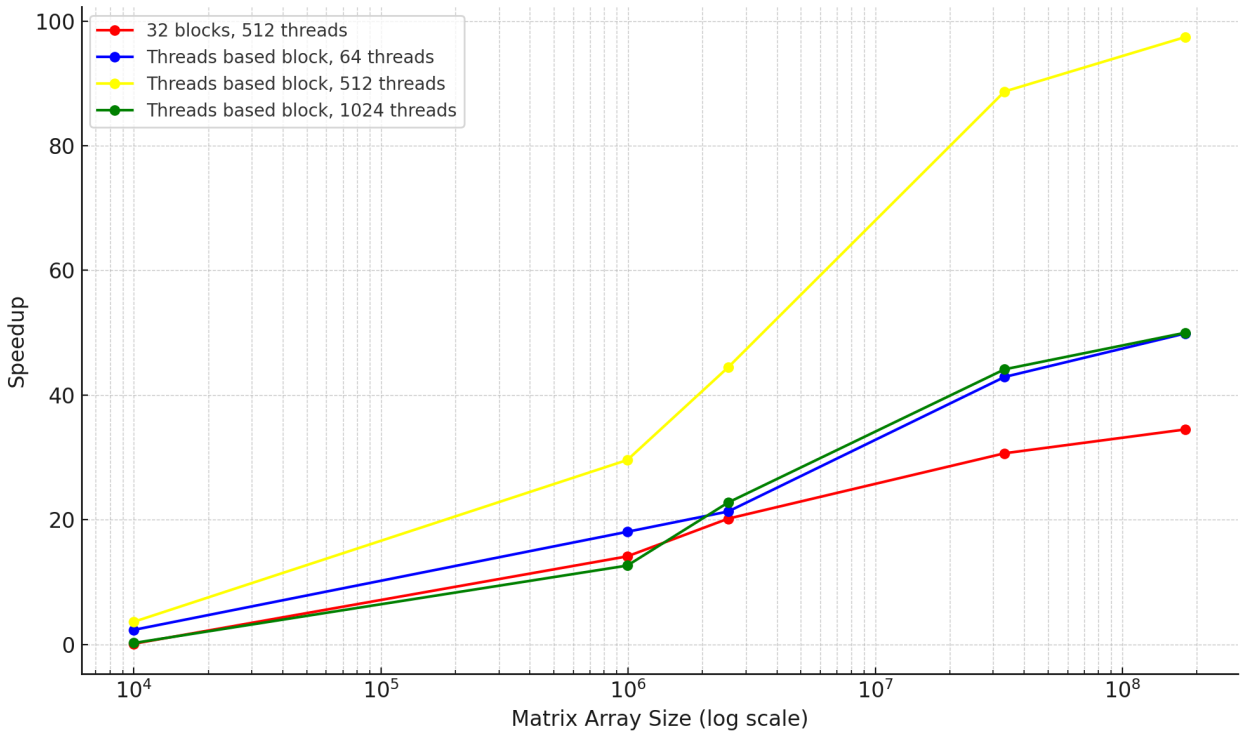


Figura 4: Gráfico de speedup por tamanho da matriz de entrada, para as quatro configurações analisadas no exercício CUDA: Matrix Sum

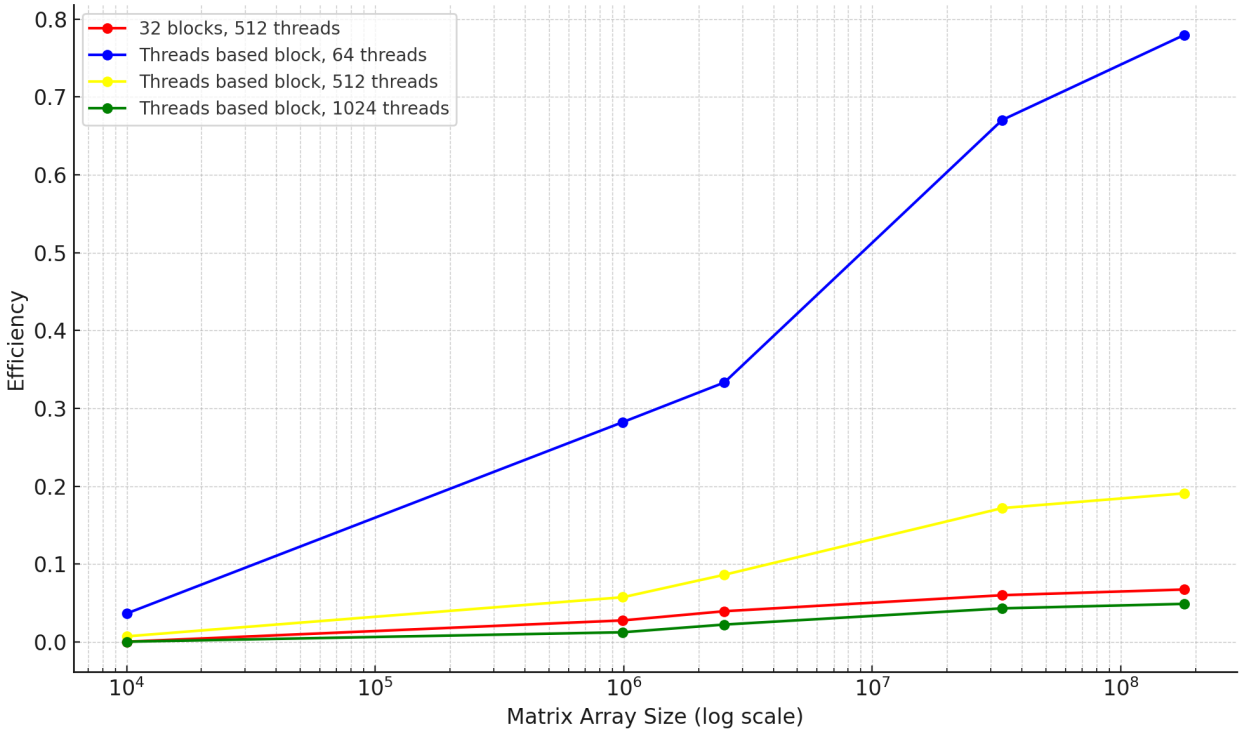


Figura 5: Gráfico de eficiência por tamanho da matriz de entrada, para as quatro configurações analisadas no exercício CUDA: Matrix Sum

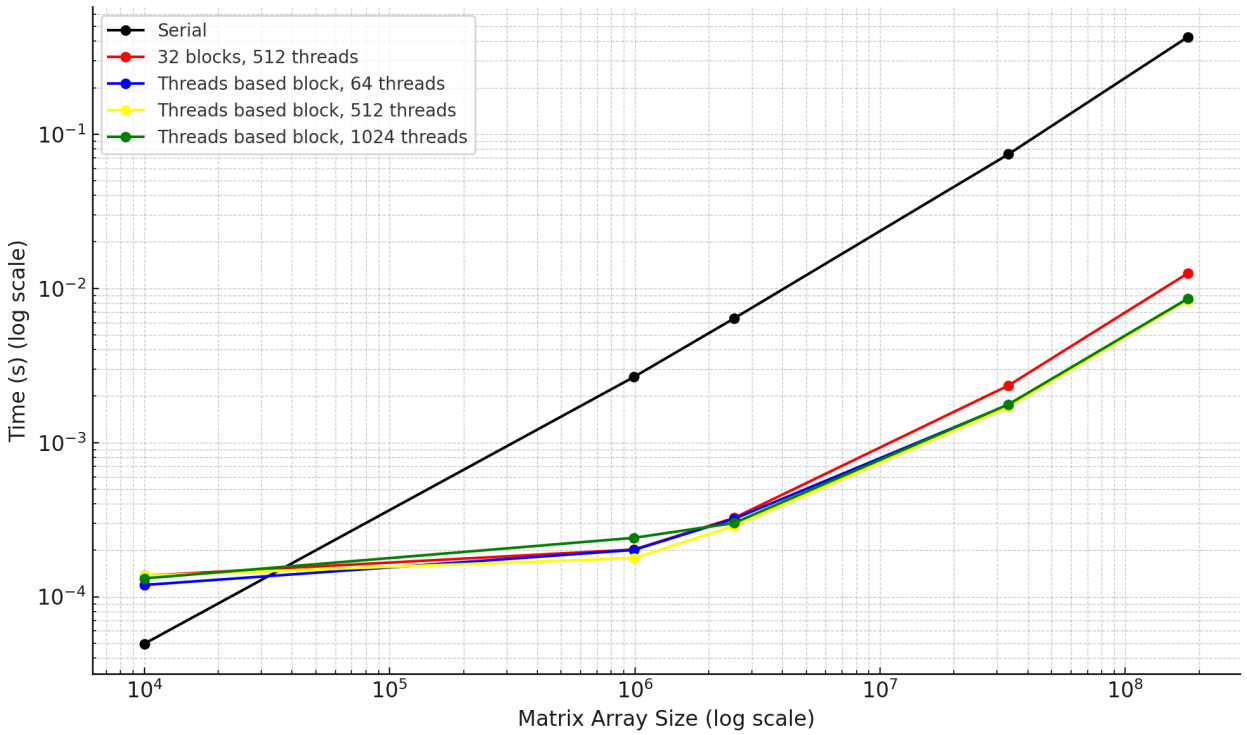
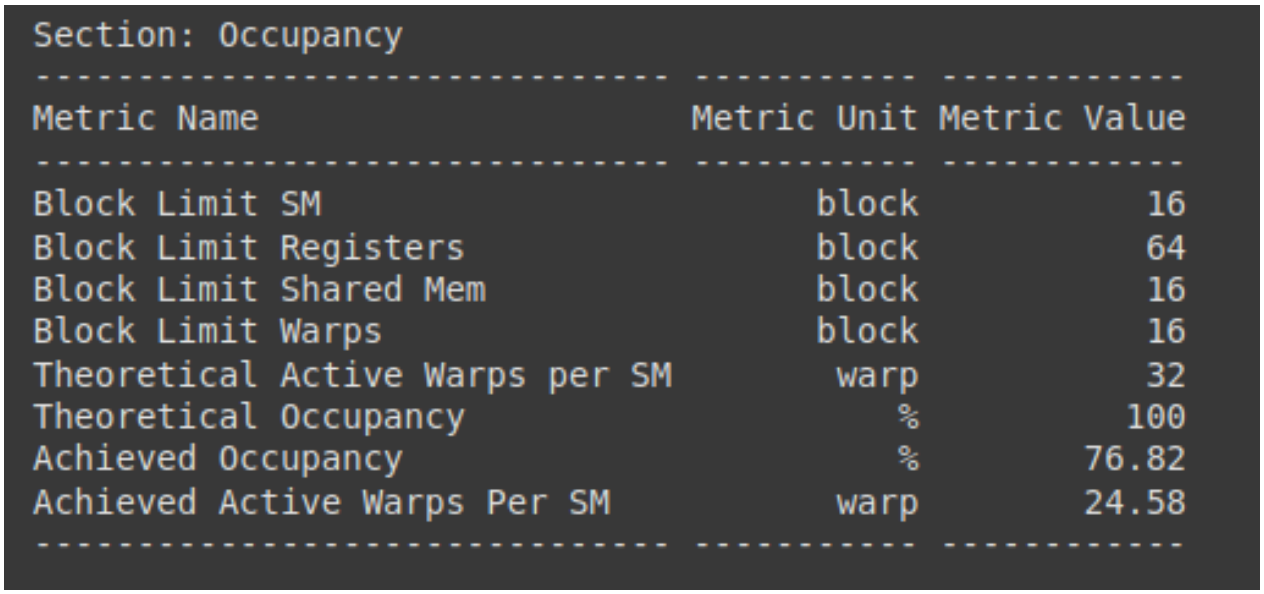


Figura 6: Gráfico de tempo de execução serial e paralela por tamanho da matriz de entrada, para as quatro configurações analisadas no exercício CUDA: Matrix Sum

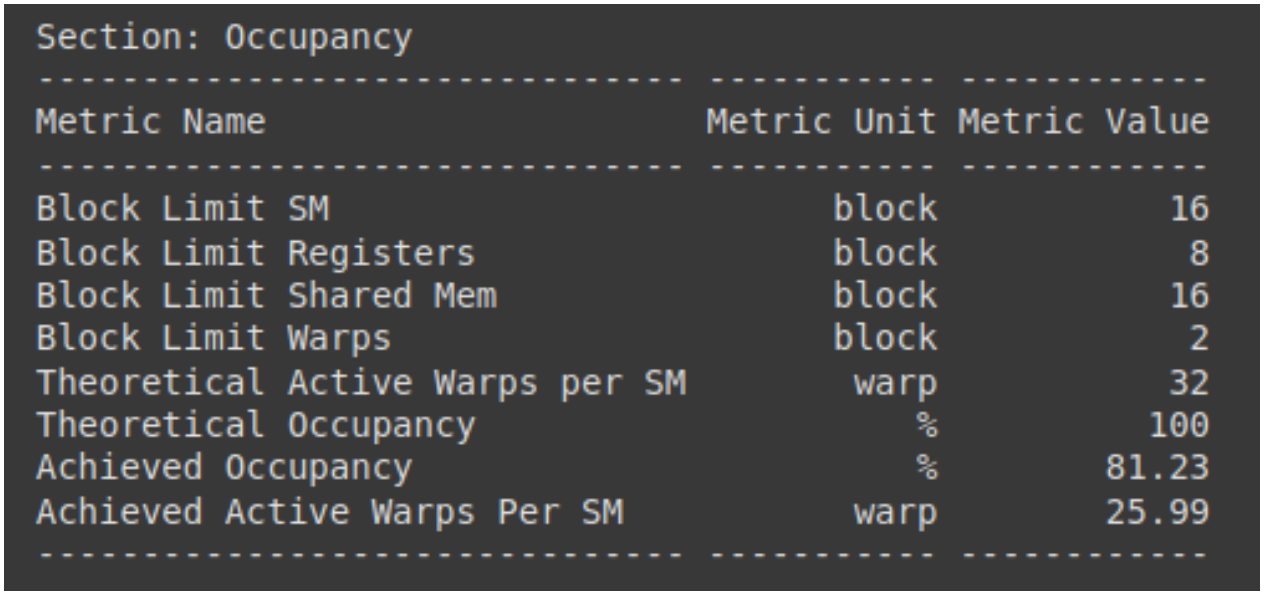
```
Section: Occupancy
------------------------------- ----------- ------------
Metric Name                     Metric Unit Metric Value
------------------------------- ----------- ------------
Block Limit SM                        block           16
Block Limit Registers                 block            8
Block Limit Shared Mem                block           16
Block Limit Warps                     block            2
Theoretical Active Warps per SM        warp           32
Theoretical Occupancy                     %          100
Achieved Occupancy                        %        49.71
Achieved Active Warps Per SM           warp        15.91
------------------------------- ----------- ------------
```

Figura 7: Resultado da análise de ocupação com Nsight Compute, para a configuração de 32 blocos e 512 threads por bloco no exercício CUDA: Matrix Sum



```
Section: Occupancy
------------------------------ ----------- ------------
Metric Name                    Metric Unit Metric Value
------------------------------ ----------- ------------
Block Limit SM                       block           16
Block Limit Registers                block           64
Block Limit Shared Mem               block           16
Block Limit Warps                    block           16
Theoretical Active Warps per SM       warp           32
Theoretical Occupancy                    %          100
Achieved Occupancy                       %        76.82
Achieved Active Warps Per SM          warp        24.58
------------------------------ ----------- ------------
```

Figura 8: Resultado da análise de ocupação com Nsight Compute, para a configuração de blocos definidos por threads e 64 threads por bloco no exercício CUDA: Matrix Sum



```
Section: Occupancy
------------------------------ ----------- ------------
Metric Name                    Metric Unit Metric Value
------------------------------ ----------- ------------
Block Limit SM                       block           16
Block Limit Registers                block            8
Block Limit Shared Mem               block           16
Block Limit Warps                    block            2
Theoretical Active Warps per SM       warp           32
Theoretical Occupancy                    %          100
Achieved Occupancy                       %        81.23
Achieved Active Warps Per SM          warp        25.99
------------------------------ ----------- ------------
```

Figura 9: Resultado da análise de ocupação com Nsight Compute, para a configuração de blocos definidos por threads e 512 threads por bloco no exercício CUDA: Matrix Sum

```
Section: Occupancy
------------------------------- ----------- ------------
Metric Name                     Metric Unit Metric Value
------------------------------- ----------- ------------
Block Limit SM                        block           16
Block Limit Registers                 block            4
Block Limit Shared Mem                block           16
Block Limit Warps                     block            1
Theoretical Active Warps per SM        warp           32
Theoretical Occupancy                     %          100
Achieved Occupancy                        %        88.56
Achieved Active Warps Per SM           warp        28.34
------------------------------- ----------- ------------
```

Figura 10: Resultado da análise de ocupação com Nsight Compute, para a configuração de blocos definidos por threads e 1024 threads por bloco no exercício CUDA: Matrix Sum

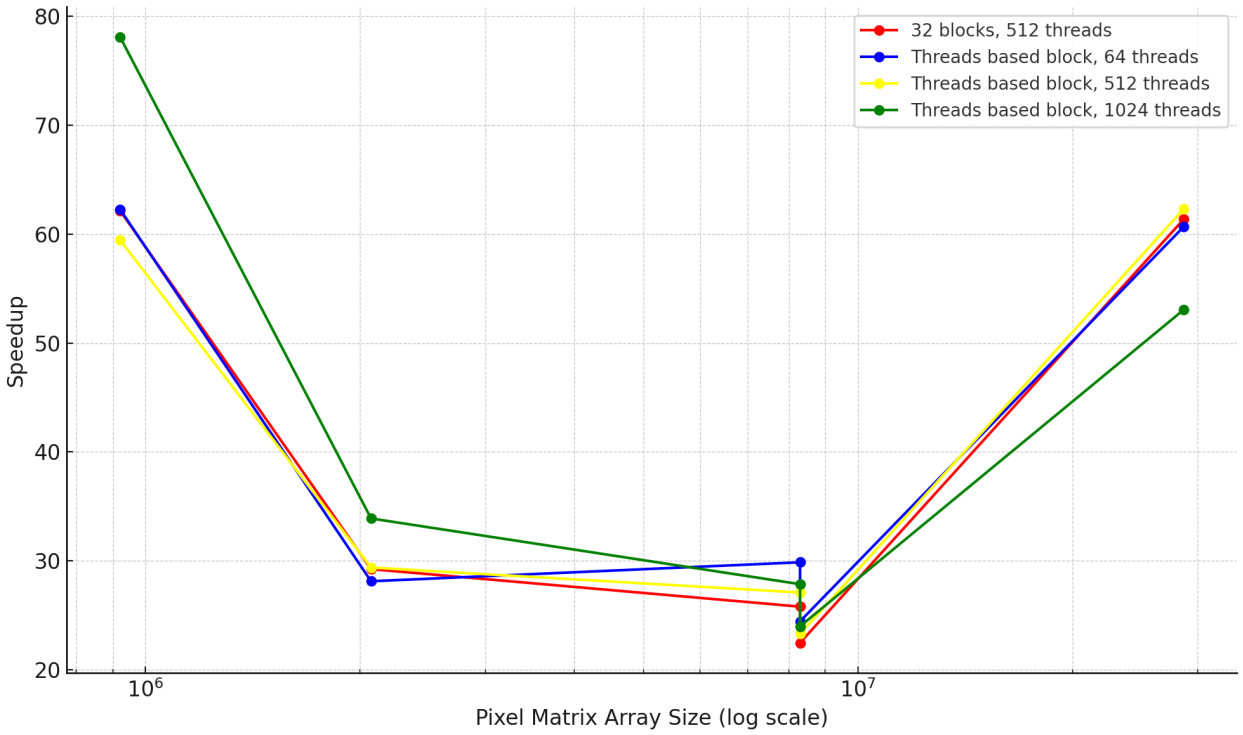


Figura 11: Gráfico de speedup por tamanho da matriz de pixels, para as quatro configurações analisadas no exercício CUDA: Histogram

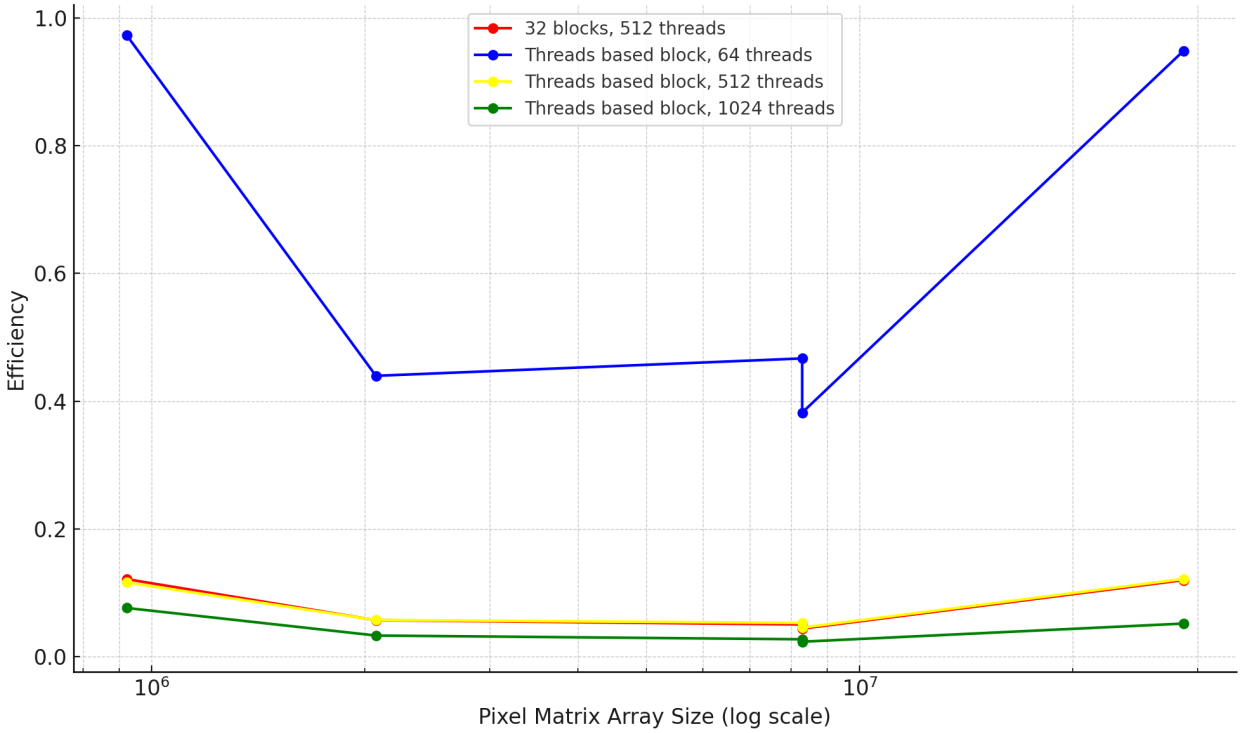


Figura 12: Gráfico de eficiência por tamanho da matriz de pixels, para as quatro configurações analisadas no exercício CUDA: Histogram

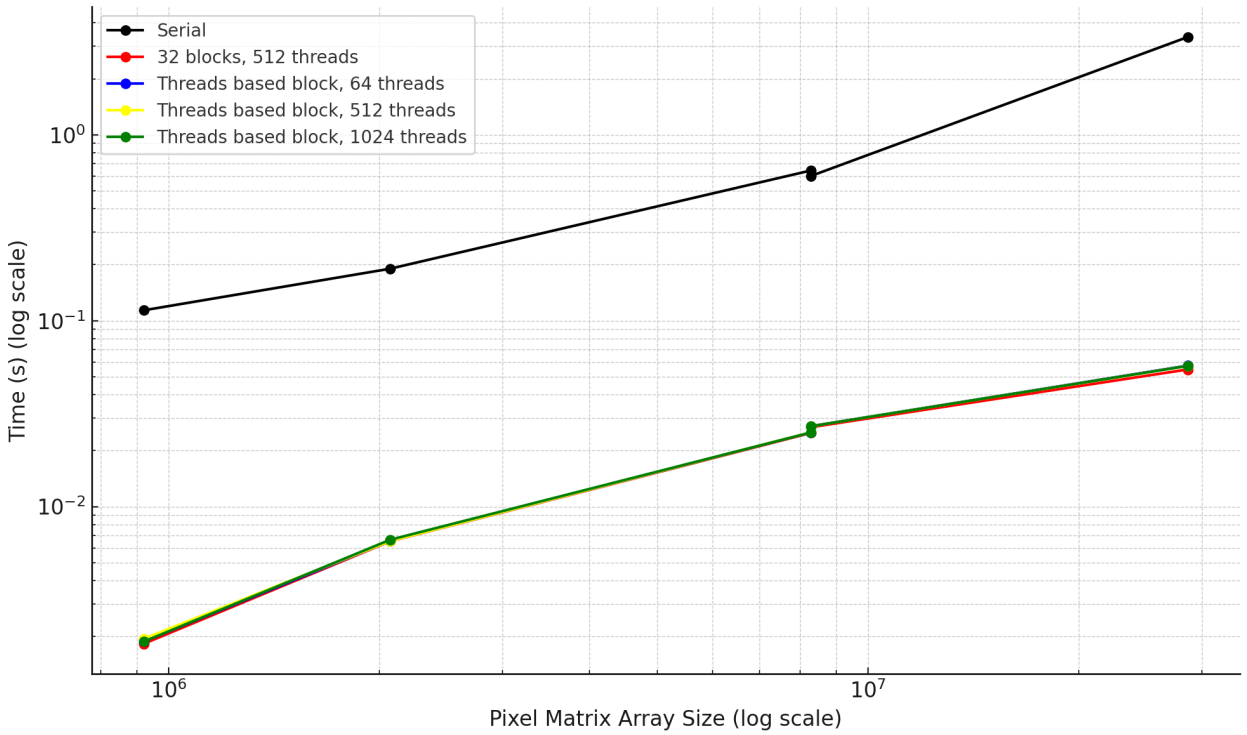
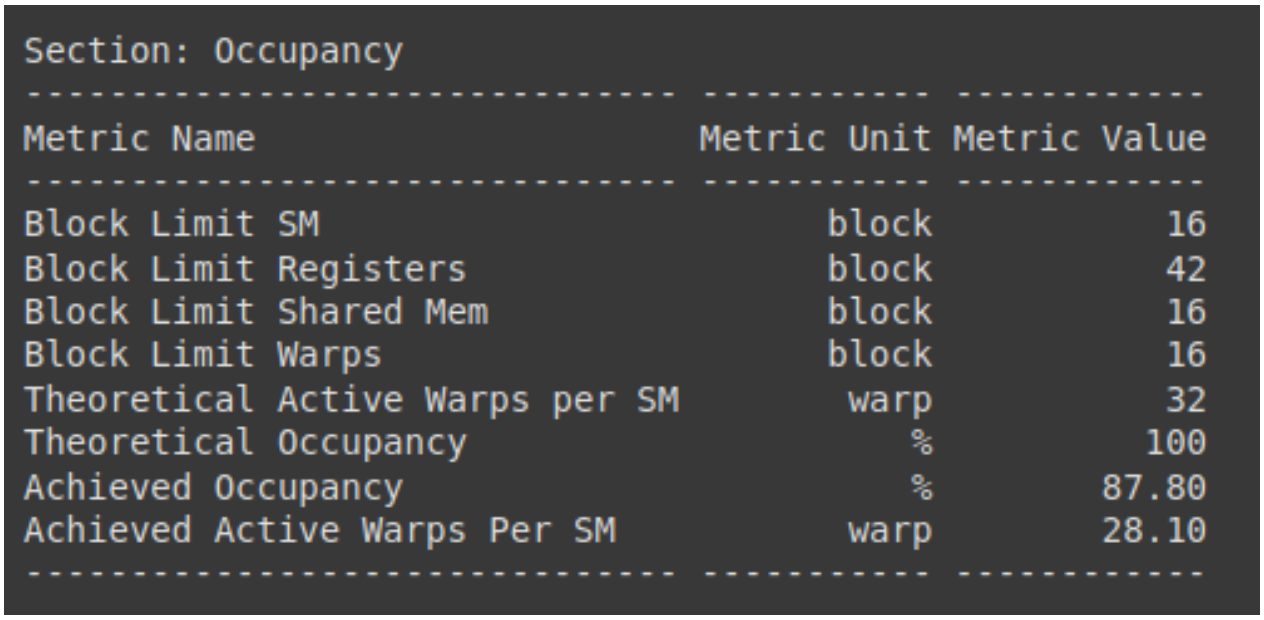


Figura 13: Gráfico de tempo de execução serial e paralela por tamanho da matriz de pixels, para as quatro configurações analisadas no exercício CUDA: Histogram

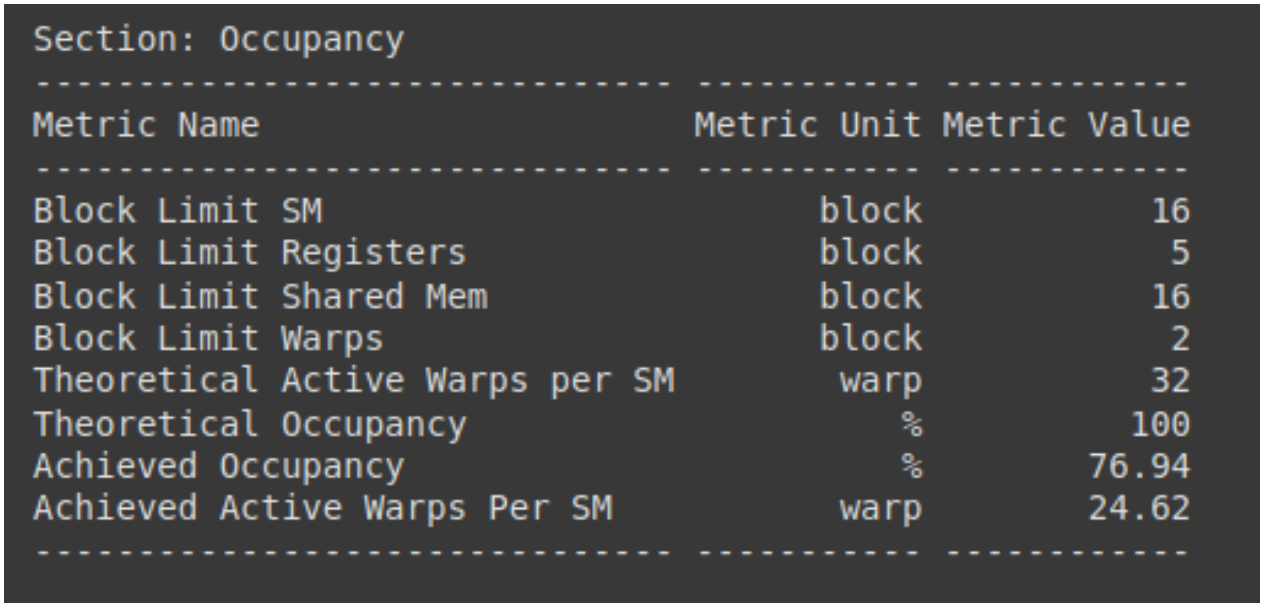
```
Section: Occupancy
------------------------------- ----------- ------------
Metric Name                     Metric Unit Metric Value
------------------------------- ----------- ------------
Block Limit SM                        block           16
Block Limit Registers                 block            5
Block Limit Shared Mem                block           16
Block Limit Warps                     block            2
Theoretical Active Warps per SM        warp           32
Theoretical Occupancy                     %          100
Achieved Occupancy                        %        49.67
Achieved Active Warps Per SM           warp        15.90
------------------------------- ----------- ------------
```

Figura 14: Resultado da análise de ocupação com Nsight Compute, para a configuração de 32 blocos e 512 threads por bloco no exercício CUDA: Histogram



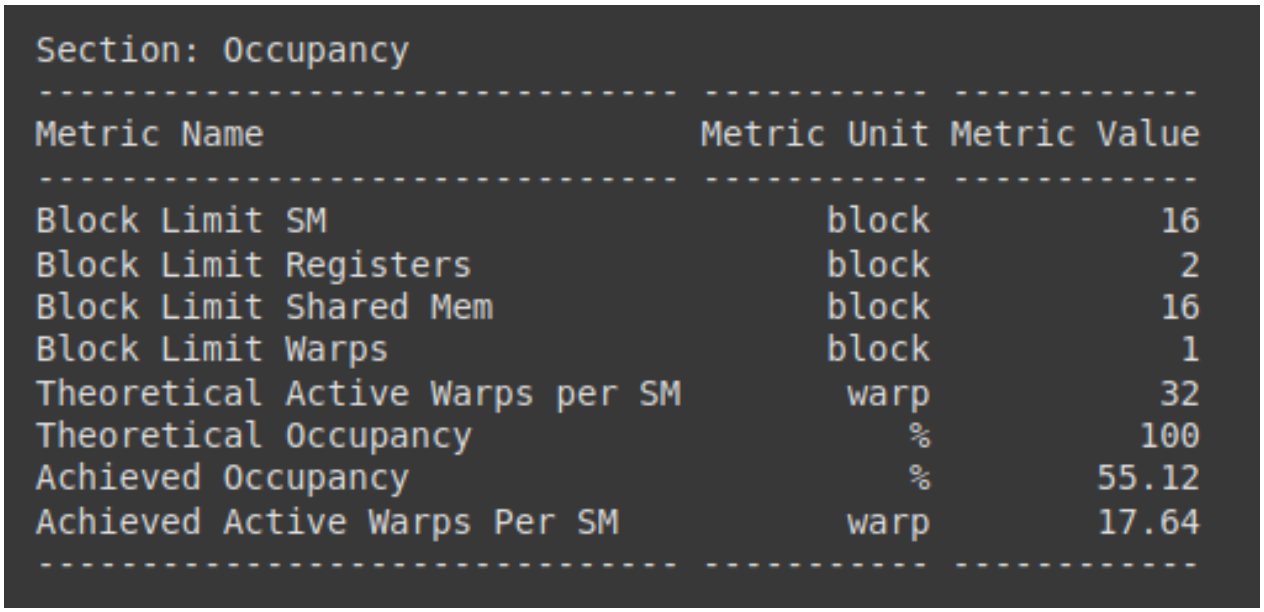
```
Section: Occupancy
------------------------------ ----------- ------------
Metric Name                    Metric Unit Metric Value
------------------------------ ----------- ------------
Block Limit SM                       block           16
Block Limit Registers                block           42
Block Limit Shared Mem               block           16
Block Limit Warps                    block           16
Theoretical Active Warps per SM       warp           32
Theoretical Occupancy                    %          100
Achieved Occupancy                       %        87.80
Achieved Active Warps Per SM          warp        28.10
------------------------------ ----------- ------------
```

Figura 15: Resultado da análise de ocupação com Nsight Compute, para a configuração de blocos definidos por threads e 64 threads por bloco no exercício CUDA: Histogram



```
Section: Occupancy
------------------------------ ----------- ------------
Metric Name                    Metric Unit Metric Value
------------------------------ ----------- ------------
Block Limit SM                       block           16
Block Limit Registers                block            5
Block Limit Shared Mem               block           16
Block Limit Warps                    block            2
Theoretical Active Warps per SM       warp           32
Theoretical Occupancy                    %          100
Achieved Occupancy                       %        76.94
Achieved Active Warps Per SM          warp        24.62
------------------------------ ----------- ------------
```

Figura 16: Resultado da análise de ocupação com Nsight Compute, para a configuração de blocos definidos por threads e 512 threads por bloco no exercício CUDA: Histogram



```
Section: Occupancy
------------------------------- ----------- ------------
Metric Name                     Metric Unit Metric Value
------------------------------- ----------- ------------
Block Limit SM                        block           16
Block Limit Registers                 block            2
Block Limit Shared Mem                block           16
Block Limit Warps                     block            1
Theoretical Active Warps per SM        warp           32
Theoretical Occupancy                     %          100
Achieved Occupancy                        %        55.12
Achieved Active Warps Per SM           warp        17.64
------------------------------- ----------- ------------
```

Figura 17: Resultado da análise de ocupação com Nsight Compute, para a configuração de blocos definidos por threads e 1024 threads por bloco no exercício CUDA: Histogram

Referências

[1] Service Not Allowed. Disponível em: <https://colab.research.google.com/notebooks/welcome.ipynb?hl=pt-BR>. Acesso em: 20 maio. 2025.

[2] NVIDIA T4 TENSOR CORE GPU SPECIFICATIONS GPU Architecture NVIDIA Turing NVIDIA Turing Tensor Cores 320 NVIDIA CUDA ® Cores 2,560. [s.l: s.n.]. Disponível em: <https://www.nvidia.com/content/dam/en-zz/Solutions/Data-Center/tesla-t4/t4-tensor-core-datasheet.pdf>. Acesso em: 20 maio. 2025.

[3] REINVENTED, G. NVIDIA TURING GPU ARCHITECTURE. [s.l: s.n.]. Disponível em: <https://images.nvidia.com/aem-dam/en-zz/Solutions/design-visualization/technologies/turing-architecture/NVIDIA-Turing-Architecture-Whitepaper.pdf>. Acesso em: 20 maio. 2025.

[4] NVIDIA Nsight Compute. [s.l: s.n.]. Disponível em: <https://developer.nvidia.com/nsight-compute>. Acesso em: 20 maio. 2025.